

# 数据库(5)

孙甲松

[sunjiasong@tsinghua.edu.cn](mailto:sunjiasong@tsinghua.edu.cn)

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 13901216180/62796193

2025.10.

# 更换教室通知

从第6周周四晚上（10月23日）开始，《数据库》课上课教室更换到：

**建华/经管新楼A420**

位于清华十字路口东南侧（新清华学堂路对面），进建华楼北门，电梯或楼梯到4层，**A420**在电梯左侧紧挨电梯（门上写着 案例室）

# 第3章 关系数据库标准语言SQL（续）

- 3.4 SQL数据查询
  - 3.4.1 简单查询
  - 3.4.2 连接查询
  - 3.4.3 嵌套查询
  - 3.4.4 集合查询
  - 3.4.5 基于派生表的查询
- 3.5 SQL数据更新
  - 3.5.1 插入
  - 3.5.2 更新
  - 3.5.3 删除
- 3.6 空值的处理
- 3.7 SQL视图
  - 3.7.1 定义视图
  - 3.7.2 删除视图
  - 3.7.3 查询视图
  - 3.7.4 更新视图
  - 3.7.5 视图的作用
- 3.8 SQL数据控制

## 3.4.2 连接查询

- 如果查询涉及到两个和两个以上关系(表), 必须使用连接操作, 故称为连接查询。包括:
  - 等值连接 (=)
  - 非等值连接 (=之外其它5个 $\theta$ 运算符)
  - 自然连接 (等值连接去掉重复属性名)
  - 自身连接
  - 外连接
  - 复合条件连接

## 3.4.2 连接查询

### 一. 等值连接查询

例18. 查全体学生的信息以及所选课程课号和成绩。

```
SELECT S.*, SC.*  
FROM S, SC  
WHERE S.SNO=SC.SNO;
```

关系1.属性1  $\theta$  关系2.属性2

大家设想一下，下面的查询会是什么结果？

```
SELECT S.*, SC.*  
FROM S, SC;
```

是S与SC的笛卡尔乘积！

## 3.4.2 连接查询

### 二. 自然连接查询

例19. 查全体学生的信息以及所选课程课号和成绩。

```
SELECT S.SNO, S.SName, S.Sex, S.Age,  
       S.Dept, SC.CNO, SC.Grade  
FROM S, SC  
WHERE S.SNO=SC.SNO;
```

如果字段（属性）不重名，则不需要加前缀：

```
SELECT S.SNO, SName, Sex, Age, Dept,  
       CNO, Grade  
FROM S, SC  
WHERE S.SNO=SC.SNO;
```

## 3.4.2 连接查询

### 三. 自身连接查询

同一个关系（表）在连接中出现一次以上，需要给同名的关系另起别名，以区分关系出现的先后。

例20. 查每一门课程的间接先修课。

```
SELECT First.CNO, Second.PCNO  
FROM C First, C Second  
WHERE First.PCNO=Second.CNO;
```

或者写成：

```
SELECT First.CNO, Second.PCNO  
FROM C As First, C As Second  
WHERE First.PCNO=Second.CNO;
```

Frist (C)

| CNO | Cname | PCNO |
|-----|-------|------|
|     |       |      |

Second (C)

| CNO | Cname | PCNO |
|-----|-------|------|
|     |       |      |

First.PCNO=Second.CNO

运行结果:

| First.CNO | Second.PCNO |
|-----------|-------------|
|           |             |
|           |             |



## 3.4.2 连接查询

### 四. 外连接查询

若一个关系的元组在连接的关系中没有等值元组，结果中将不出现。

例21. 查全体学生的信息以及所选课程课号和成绩。

```
SELECT S.SNO, SName, Sex, Age, Dept,  
       CNO, Grade  
FROM S, SC  
WHERE S.SNO=SC.SNO(*);
```

某学生未选课，则结果中CNO, Grade列为空。

根据出现位置，外连接分为：

右外连接

左外连接

## 3.4.2 连接查询

另一种写法:

```
SELECT S.SNO, SName, Sex, Age, Dept,  
       CNO, Grade  
FROM S LEFT OUTER JOIN SC  
       ON (S.SNO=SC.SNO);
```

又另一种写法: (在SNO上进行左连接)

```
SELECT S.SNO, SName, Sex, Age, Dept,  
       CNO, Grade
```

```
FROM S LEFT OUTER JOIN SC USING (SNO);
```

说明: 左外连接列出左边关系 (S) 中所有元组, 而右外连接列出右边关系 (SC) 中所有元组。

## 3.4.2 连接查询

五. 复合条件连接查询 及 多表连接查询

WHERE条件中出现多个查询条件。

例22. 求选修C1课且成绩80分以上的学生学号，姓名及其成绩。

```
SELECT S.SNO, SName, Grade
FROM S, SC
WHERE S.SNO=SC.SNO and CNO='C1'
      and Grade>=80;
```

查询条件中S.SNO=SC.SNO实现S与SC两表之间的自然连接。

## 3.4.2 连接查询

例23. 求成绩90分以上的学生学号、姓名、课名及其成绩。

```
SELECT S.SNO, SName, Cname, Grade  
FROM S, C, SC  
WHERE S.SNO=SC.SNO and  
       C.CNO=SC.CNO and Grade >=90;
```

查询条件中：

S.SNO=SC.SNO实现S与SC两表之间的自然连接，  
C.CNO=SC.CNO实现C与SC两表之间的自然连接，  
最终实现S， C， SC三表之间的自然连接。

### 3.4.3 嵌套查询

又称为子查询，指SELECT-FROM-WHERE结构的嵌套一. IN ( $\in$ )谓词子查询

例24. 求选修了课名为C++的学生学号、姓名。

```
SELECT SNO, Sname
FROM S
WHERE SNO IN
  ( SELECT SNO
    FROM SC
    WHERE CNO IN
      ( SELECT CNO
        FROM C
        WHERE Cname='C++'));
```

### 3.4.3 嵌套查询

IN( $\epsilon$ )谓词是多值的，即集合运算，属于不相关子查询。  
由里向外逐层完成，因此，子查询：

```
SELECT CNO  
FROM C  
WHERE Cname='C++';
```

先独立完成。

上面的例子还可以写成：

```
SELECT SNO, Sname  
FROM S, SC, C  
WHERE S.SNO=SC.SNO and SC.CNO=C.CNO  
and Cname='C++';
```

同一个查询，可以有多种实现方法，但执行效率不同。

### 3.4.3 嵌套查询

#### 二. 带比较运算符的嵌套子查询

若是单值(结果), 可用比较运算符  $=, !=, >, >=, <, <=, <>, !>, !<$  等。

例25. 求与张三同系的学生学号、姓名、系别。

```
SELECT SNO, Sname, Dept
FROM S
WHERE Dept =
  ( SELECT Dept
    FROM S
    WHERE Sname='张三');
```

### 3.4.3 嵌套查询

注意：下列写法是错误的！！！！

```
SELECT SNO, Sname, Dept
FROM S
WHERE ( SELECT Dept
        FROM S
        WHERE Sname='张三') = Dept;
```

子查询必须跟在比较符之后！现在不清楚这一条限制是否取消了。



### 3.4.3 嵌套查询

例26. 求C1课高于平均成绩的学生学号和成绩。

```
SELECT SNO, Grade
FROM SC
WHERE Grade >=
  ( SELECT AVG(Grade)
    FROM SC
    WHERE CNO='C1') and CNO='C1';
```

再次强调：子查询必须要出现在比较符之后，写为：

```
SELECT SNO, Grade
FROM SC
WHERE Grade>=AVG(Grade) and CNO='C1';
```

是完全错误的！集函数不能直接出现在where中。

## 3.4.3 嵌套查询

### 三. 带有ANY、ALL谓词的子查询

比较运算符必须是单值，对于多值情况，可以使用ANY或ALL

- **>ANY** 大于子查询结果中某个值（大于一个即为真）
- **<ANY** 小于子查询结果中某个值（小于一个即为真）
- **>ALL** 大于子查询结果中所有值（有一个不大于则为假）
- **<ALL** 小于子查询结果中所有值（有一个不小于则为假）
- .....
- **>=, <=, =, !=**(或写为<>) 等组合

### 3.4.3 嵌套查询

例27. 查非电子系中比电子系某一个学生年龄小的学生学号、姓名、年龄、系别。

```
SELECT SNO, Sname, Age, Dept  
FROM S
```

```
WHERE Age < ANY
```

```
( SELECT distinct Age  
  FROM S
```

```
WHERE Dept='EE') and Dept != 'EE';
```

注意： 1. 这些谓词效率不高。

2. Age<ANY 可等价于 Age<MAX(Age)

3. Age<ALL 可等价于 Age<MIN(Age)

### 3.4.3 嵌套查询

上面的例27可改写为:

```
SELECT SNO, Sname, Age, Dept
FROM S
WHERE Age <
    ( SELECT MAX(Age)
      FROM S
      WHERE Dept='EE') and Dept != 'EE';
```

## 3.4.3 嵌套查询

### 四. EXISTS ( $\exists$ )谓词子查询

EXISTS即存在量词  $\exists$ ，执行结果是逻辑值：true/false

例28. 求选修了C1课的学生学号、姓名。

```
SELECT SNO, Sname
FROM S
WHERE EXISTS
( SELECT *
  FROM SC
  WHERE SC.SNO=S.SNO and CNO='C1');
```

注意： EXISTS是**相关子查询**(Correlated Subquery)。

WHERE查询条件中需要有关系的**连接操作**。

上例中，是取S中每一行(元组)去执行()中子查询，为真则写出SNO, Sname。

### 3.4.3 嵌套查询

例29. 求未选修C1课的学生学号、姓名。

```
SELECT SNO, Sname
FROM S
WHERE NOT EXISTS /* 不存在 */
( SELECT *
  FROM SC
  WHERE SC.SNO=S.SNO and CNO='C1');
```

SQL中没有全称谓词 $\forall$  (For All)和蕴涵谓词 $\rightarrow$  (Implication)

$$(\forall x)P \equiv \neg(\exists x(\neg P))$$

$$A \rightarrow B \equiv \neg A \vee B$$

### 3.4.3 嵌套查询

证明:  $A \rightarrow B \equiv \neg A \vee B$       T=true    F=false

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| T | T | T | F | T | T | T |
| T | F | F | F | T | F | F |
| F | T | T | T | F | T | T |
| F | T | F | T | F | T | F |

在全值域范围内,  $A \rightarrow B$  和  $\neg A \vee B$  的结果相同,  
因此  $A \rightarrow B$  恒等于  $\neg A \vee B$ , 所有  $A \rightarrow B$  问题可  
以通过求解验证  $\neg A \vee B$  来实现。

### 3.4.3 嵌套查询

例30. 求选修了全部课程的的学生学号、姓名。

```
SELECT SNO, Sname
FROM S
WHERE NOT EXISTS
  ( SELECT *
    FROM C
    WHERE NOT EXISTS
      (SELECT *
       FROM SC
       WHERE SC.SNO=S.SNO and SC.CNO=C.CNO));
```

$(\forall x)P \equiv \neg(\exists x(\neg P))$  不存在他没选修的课程。



### 3.4.3 嵌套查询

例31. 求选修了学号S1学生所选全部课程的学生学号。

设：p为学生S1选课y，q为学生x选课y

则问题可以表示为：  $(\forall y)(p \rightarrow q)$

$$\begin{aligned}(\forall y)(p \rightarrow q) &\equiv \neg(\exists y(\neg(p \rightarrow q))) \\ &\equiv \neg(\exists y(\neg(\neg p \vee q))) \\ &\equiv \neg(\exists y(p \wedge \neg q))\end{aligned}$$

语义：不存在这样的课程y，S1选修了，而x学生没选修。  
因此x学生是选修了学号S1学生所选全部课程的学生。

### 3.4.3 嵌套查询

```
SELECT SNO  
FROM SC X  
WHERE NOT EXISTS  
  ( SELECT *  
    FROM SC Y  
    WHERE Y.SNO='S1' and NOT EXISTS  
      ( SELECT *  
        FROM SC Z  
        WHERE Z.SNO=X.SNO and Z.CNO=Y.CNO));
```

## 3.4.4 集合查询

例32. 求选修了学号S1和S2学生所选全部课程的学生学号。

```
SELECT SNO
FROM SC X
WHERE NOT EXISTS
( SELECT *
  FROM SC Y
  WHERE (Y.SNO='S1' OR Y.SNO='S2') and
    NOT EXISTS
    ( SELECT *
      FROM SC Z
      WHERE Z.SNO=X.SNO and Z.CNO=Y.CNO));
```

## 3.4.4 集合查询

并操作 UNION

交操作 INTERSECT

差操作 MINUS

注意：许多数据库系统不支持这些操作，因此不提倡使用。

例33. 求电子系或年龄小于18岁的学生信息。

```
SELECT *  
FROM S  
WHERE Dept='EE'
```

UNION

```
SELECT *  
FROM S  
WHERE Age<18;
```

等价于： 

```
SELECT *  
FROM S  
WHERE Dept='EE' OR Age<18;
```

### 3.4.4 集合查询

例34. 求选修了C1课或选修了C2课的学生学号。

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C1'
```

```
UNION
```

(不提倡使用!)

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C2';
```

等价于:

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C1' OR CNO='C2';
```

### 3.4.4 集合查询

例35. 求既选修了C1课又选修了C2课的学生学号。

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C1'
```

```
INTERSECT      (不提倡使用! )
```

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C2';
```

等价于: SELECT SNO

```
FROM SC
```

```
WHERE CNO='C1' and SNO in
```

```
( SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C2');
```

## 3.4.4 集合查询

例36. 求只选修了C1课而未选修C2课的学生学号。

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C1'
```

```
MINUS      (不提倡使用! )
```

```
SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C2';
```

等价于: SELECT SNO

```
FROM SC
```

```
WHERE CNO='C1' and SNO NOT in
```

```
( SELECT SNO
```

```
FROM SC
```

```
WHERE CNO='C2');
```

## 3.4.5 基于派生表的查询

子查询不仅可以出现在WHERE子句中，还可以出现在FROM子句中，此时子查询生成的临时派生表（**derived table**）成为主查询的查询对象。这是近些年SQL国际标准新引入的。

例37. 找出每个学生超过他自己选修课程平均成绩的课程号。

```
SELECT SNO,CNO
FROM SC, (SELECT SNO, AVG(Grade) FROM SC
          GROUP BY SNO) As Avg_SC(ASNO,AGrade)
WHERE SC.SNO=Avg_SC.ASNO
      AND SC.Grade >= Avg_SC.AGrade;
```

这里子查询 `SELECT SNO,AVG(Grade) FROM SC GROUP BY SNO` 将生成一个派生表 `Avg_SC(ASNO,AGrade)`。

这其中的 **As** 可以省略。



## 3.4.5 基于派生表的查询

需要说明的是，这种生成派生表的查询不是必须的，可以由别的查询方式替代。

比如上例可以用嵌套子查询替代：

```
SELECT SNO,CNO  
FROM SC X  
WHERE Grade >= (SELECT AVG(Grade)  
                  FROM SC Y  
                  WHERE Y.SNO=X.SNO);
```

## 3.5 SQL数据更新

插入 INSERT

删除 DELETE

修改 UPDATE

### 3.5.1 插入数据

语句格式: INSERT INTO 表名[(属性名1[,属性名2]...)]  
VALUES(常量1[,常量2]...);

一次一元组插入。

或: INSERT INTO 表名[(属性名1[,属性名2]...)]  
[子查询];

批量插入元组。

## 3.5 SQL数据更新

例1. 单记录插入。

```
INSERT INTO S VALUES('S1', '张三', 'M', 18, 'EE');
```

例2. 单记录部分插入。

```
INSERT INTO SC(SNO, CNO) VALUES('S1', 'C1');
```

例3. 多记录插入。求每个系的平均年龄，并保存起来。

```
CREATE TABLE DeptAge (  
    Dept char(15),  
    AVGAge float );
```

```
Insert into DeptAge (Dept, AVGAge)  
    SELECT Dept, AVG(Age)  
    FROM S  
    GROUP BY Dept ;
```

## 3.5 SQL数据更新

### 3.5.2 更新数据

语句格式: UPDATE 表名

SET 属性1=表达式1 [, 属性2=表达式2]...

[FROM 表名1[,表名2]...

[WHERE 条件表达式];

进行单个或批量记录修改。

UPDATE只能同时修改一个关系(表), 若修改涉及到多个关系, 则需要多个UPDATE语句分别修改, 以保持数据的一致性。

## 3.5 SQL数据更新

例4. 单记录修改。将学号S1的同学改名为张三。

```
UPDATE S  
SET Sname='张三'  
WHERE SNO='S1';
```

例5. 多记录修改。将C1课所有学生成绩加5分。

```
UPDATE SC  
SET Grade=Grade+5  
WHERE CNO='C1';
```

## 3.5 SQL数据更新

例6. 多记录修改。将MA系学生所有学生成绩置为0。

```
UPDATE SC
SET Grade=0
WHERE 'MA' =
( SELECT Dept
  FROM S
   WHERE S.SNO=SC.SNO);
```

或写为:

```
UPDATE SC
SET Grade=0
FROM S
WHERE S.SNO=SC.SNO AND S.Dept='MA';
```

## 3.5 SQL数据更新

### 3.5.3 删除数据

语句格式: **DELETE FROM 表名**  
**[WHERE 条件表达式];**

单个或批量记录删除。

例7. 单记录删除。删除学号S9同学的信息。

```
DELETE FROM S WHERE SNO='S9';
```

例8. 多记录删除(全部删除)。将学生表S删空。

```
DELETE FROM S;
```

例9. 带子查询的删除。删除CS系学生的选课。

```
DELETE FROM SC  
WHERE 'CS' =  
(SELECT Dept  
FROM S  
WHERE S.SNO=SC.SNO);
```

## 3.6 空值的处理

**空值**（**NULL**）是数据库中一种比较特殊的情况，含有不确定性。需要特殊处理。

数据库中某些字段在一定情况下取空值，有以下几种情况：

- (1) 应该有值，但目前还不知道属性的具体值，比如成绩。
- (2) 该属性不应该有值。比如缺考的学生，不能直接认定为0。
- (3) 由于某种原因不便于填写，比如为了保密。

### 1. 空值的产生

例如：某学生选某课后，向SC表插入一个选课元组：

```
Insert into SC Values('2024018888', 'C1', NULL);
```

### 2. 空值的判断

用 **IS NULL** 或 **IS NOT NULL** 判断属性的值是否为空。

例如：找出全部缺考的学生学号和课程号。

```
SELECT SNO,CNO FROM SC WHERE Grade IS NULL;
```



## 3.6 空值的处理

### 3. 空值的约束条件

生成基本表时，属性定义约束条件为 **NOT NULL** 的不能取空值，加了**UNIQUE**或者定义为主码的属性也不能取空值。**SQL Server**限定，只有属性定义约束条件为**NULL**的才能取空值。

### 4. 空值的算术运算、比较运算和逻辑运算

- 空值与另一个值（包括另一个空值）的算术运算的结果为空值。
- 空值与另一个值（包括另一个空值）的比较运算的结果为 **Unknown** 。
- 逻辑运算除了传统的**True**和**False**，多了一个**Unknown**，成了三值逻辑。三值逻辑的真值表可以参考下页的表3.7。

在查询语句中，只有使**WHERE**和**HAVING**子句中条件为**True**的元组才会出现在最后结果中。

有了 UNKNOWN 后，传统的逻辑运算中二值(TRUE, FALSE)逻辑就扩展成了三值逻辑。逻辑运算 AND、OR、NOT 的真值表如表 3.7 所示，其中 T 表示 TRUE，F 表示 FALSE，U 表示 UNKNOWN。

表 3.7 逻辑运算 AND、OR、NOT 真值表

| $x$ | $y$ | $x \text{ AND } y$ | $x \text{ OR } y$ | NOT $x$ |
|-----|-----|--------------------|-------------------|---------|
| T   | T   | T                  | T                 | F       |
| T   | U   | U                  | T                 | F       |
| T   | F   | F                  | T                 | F       |
| U   | T   | U                  | T                 | U       |
| U   | U   | U                  | U                 | U       |
| U   | F   | F                  | U                 | U       |
| F   | T   | F                  | T                 | T       |
| F   | U   | F                  | U                 | T       |
| F   | F   | F                  | F                 | T       |

## 3.7 SQL视图

- 视图 (**view**) 是从一个或多个基本表或视图导出的表。
- 可定义若干视图，像基本表一样操作，虽然有一些限定。
- 视图是虚表，数据库系统只在数据字典中存视图的定义。

### 3.7.1 定义视图

语句格式: **CREATE VIEW** 视图名[(属性名1[,属性名2]...)]  
**AS** 子查询 [**WITH CHECK OPTION**];

**WITH CHECK OPTION**是指对视图进行UPDATE, INSERT, DELETE 应满足的完整性限制条件。

- 视图的属性名要么全部指定，要么全部省略，直接用子查询的属性。但以下情况必须指定视图的属性名：
  - (1) 某个列是函数名或表达式（本身没有属性名）；
  - (2) 多表时有同名的属性；
  - (3) 想在视图中给某个属性用一个更好的名字。

## 3.7.1 定义视图

例1. 建立电子系学生视图EE\_S(SNO, Sname, Sex, Age)。

```
CREATE VIEW EE_S  
AS SELECT SNO, Sname, Sex, Age  
FROM S  
WHERE Dept='EE';
```

例2. 将每个学生的学号和平均成绩生成一个视图  
S\_G(SNO, AVG\_Grade)

```
CREATE VIEW S_G(SNO, AVG_Grade)  
AS SELECT SNO, AVG(Grade)  
FROM SC  
GROUP BY SNO;
```

## 3.7.2 删除视图

语句格式: DROP VIEW 视图名;

例3. DROP VIEW EE\_S;      删除电子系学生视图  
      DROP VIEW S\_G;      删除学号平均成绩视图

## 3.7.3 查询视图

查询方式与基本表一样。但实际上有一个转换过程，视图消解

例4. 查询电子系视图中年龄小于18岁的学生。

SELECT SNO,AGE

FROM EE\_S

WHERE AGE<18



SELECT SNO, AGE

FROM S

WHERE Dept='EE' and AGE<18;

### 3.7.3 查询视图

例5. 查询学生平均成绩视图中平均成绩90分以上的学生。

```
SELECT *  
FROM S_G  
WHERE AVG_Grade >=90;
```



```
SELECT SNO, AVG(Grade)  
FROM SC  
WHERE AVG(Grade) >= 90  
GROUP BY SNO;
```

集函数出现在WHERE中，错误！！！上面的查询无法执行。

正确查询应该是：

```
SELECT SNO, AVG(Grade)  
FROM SC  
GROUP BY SNO  
HAVING AVG(Grade) >= 90;
```

## 3.7.4 更新视图

通过视图插入INSERT、删除DELETE、修改UPDATE数据。

与视图的查询一样，对视图的更新最终转换成对基本表的更新。

例6. 将电子系学生视图中学号为S1的学生改名为李四。

```
UPDATE EE_S  
SET Sname='李四'  
WHERE SNO='S1';
```



```
UPDATE S  
SET Sname='李四'  
WHERE Dept='EE' and SNO= 'S1';  
更新没有错误！
```

## 3.7.4 更新视图

例7. 将学生平均分视图中学号为S1的学生平均成绩改为90。

```
UPDATE S_G  
SET AVG_Grade=90  
WHERE SNO='S1';
```

将出现错误！因为数据库中从来就没存过平均成绩。

行列子集视图：视图从单个基本表导出，而且只是去掉了某些行列（不包括码）。

理论上可更新视图：理论上可更新，但不是行列子集视图。

确切特征待进一步研究。

理论上不可更新视图：理论上已经确定不可更新的视图。

目前商用的关系数据库系统支持对行列子集视图的更新。



行列子集  
视图

理论上可更新视图

所有视图

## 3.7.4 更新视图

对更新视图的限制条件是：

1. 若视图的字段来自字段表达式或常数，则不允许对其执行 **INSERT**和**UPDATE**，但可以对其**DELETE**。
2. 若视图的字段来自集函数，则此视图不允许更新。
3. 若视图的定义中有**GROUP BY**子句，则此视图不允许更新。
4. 若视图的定义中有**DISTINCT**，则此视图不允许更新。
5. 若视图的定义中有**嵌套查询**，并且嵌套查询的**FROM**子句中涉及的表也是导出该视图的基本表，则此视图不允许更新。
6. 若视图是由两个以上基本表导出的，则此视图不允许更新。
7. 若视图是定义在一个不允许更新的视图上，则此视图不允许更新。

不可更新的视图：理论上证明不可更新，将来也不可能更新。

不允许更新的视图：现在的系统由于各种原因，不支持更新。  
将来可能允许更新。

## 3.7.4 更新视图

例8. 限制条件5的示例（高于平均成绩的选修视图）。

```
CREATE VIEW HIGH_SC  
AS SELECT SNO, CNO, Grade  
FROM SC  
WHERE Grade >  
      (SELECT AVG(Grade)  
       FROM SC);
```

## 3.7.5 视图的作用

视图的优点：

1. 简化了用户观点。
2. 视图机制使得不同的用户能以不同的方式看待同一数据。
3. 视图对于数据库的重构提供了一定程度的逻辑独立性。
4. 视图对于机密和敏感数据提供了自动的安全保护功能。
5. 适当利用视图可以更清晰地表达查询。

## 3.8 SQL控制功能

控制用户对数据的存取权力 (Privilege)。

由DBA(DataBase Administrator)数据库管理员对某一个或某一类用户授权。

**GRANT** 授权

**REVOKE** 撤消

系统将授权存入数据字典(DD)，用户操作时首先查询DD进行权限检查，以便执行或拒绝执行。

授权语句格式：

**GRANT** 权力1[,权力2]... [ON 对象类型 对象名]

**TO** 用户名1[, 用户名2]...

[WITH GRANT OPTION];

权力包括：All Privileges , SELECT, INSERT, DELETE, UPDATE 等

**WITH GRANT OPTION**: 权力是否可再授权给别人

## 3.8 SQL控制功能

例1. GRANT UPDATE(Sname), SELECT ON TABLE S  
TO WangPing;

WangPing有权查询S表，并可以修改S表中的Sname。

例2. GRANT All Privileges ON TABLE S,C,SC TO Zhang;  
Zhang有全权（四种以上的权力）访问S,C,SC表。

例3. GRANT SELECT ON TABLE S TO Sun  
WITH GRANT OPTION;

Sun有权查询S表，并可以将其权力再授权给别人。

Sun可以发授权命令：

GRANT SELECT ON TABLE S TO SunJia;

## 3.8 SQL控制功能

### 收回权力

语句格式:

```
REVOKE 权力1[,权力2]... [ON 对象类型 对象名]  
FROM 用户名1[, 用户名2]...;
```

例4. REVOKE UPDATE(Sname), SELECT ON TABLE S  
FROM WangPing;

撤消WangPing查询S表、修改S表中的Sname的权力。

例5. REVOKE All Privileges ON TABLE S,C,SC  
FROM Zhang;

撤消Zhang访问S,C,SC表的全权。

例6. REVOKE SELECT ON TABLE S FROM Sun;

撤消Sun对S表的查询权。

# 第3次作业

- 用SQL写查询:
- (i) 教材第5版 P.130 第6版 P.123
- 5. (1) (2) (3) (4) (5) (6) (7)
- (ii) 第2次作业补充题
- (1) (2) (3) (4) (5)



## 第2次作业补充题:

(在 教材第5版 P. 70 第6版 P. 64

第6题的SPJ数据库的关系模式上)

- (1) 求为工程J2供应P3零件，且数量不少于100件的供应商号。
- (2) 求供应红色、重量12以上的零件供应商姓名。
- (3) 求至少使用了供应商s1和s2供应的全部零件的工程名。
- (4) 求项目在北京，且至少有一个供应商也在北京的工程名。
- (5) 求同时使用了螺栓、螺母的项目名。